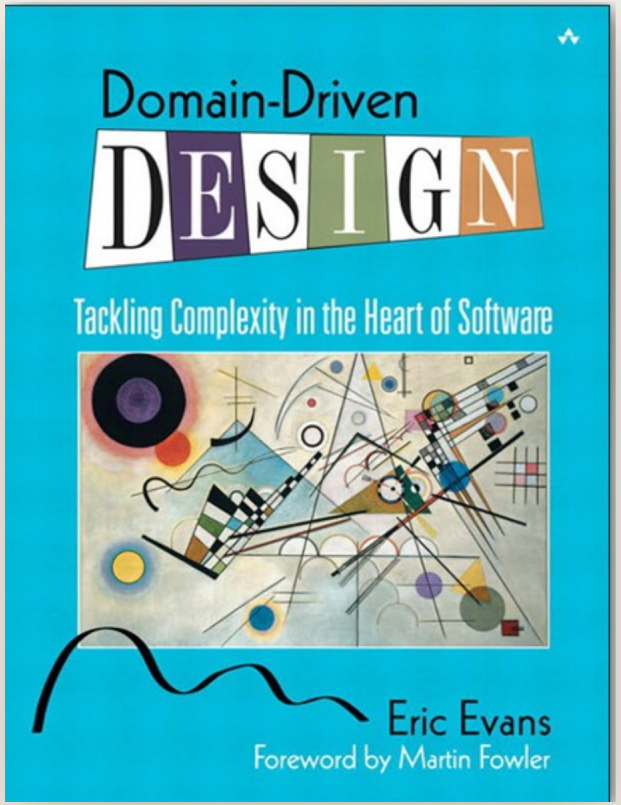
Why Ubiquitous Language is the Only Source Code You'll Commit in 2030

Virgilio Lino

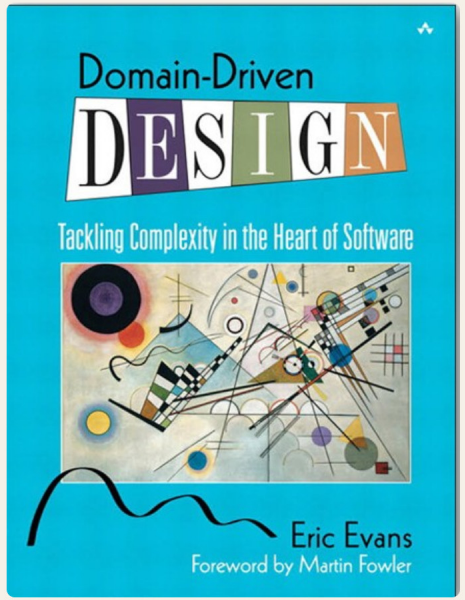
30 YEARS OF BUILDING SOFTWARE

Three AHA moments.

The second one: the blue book.



A revolutionary shift



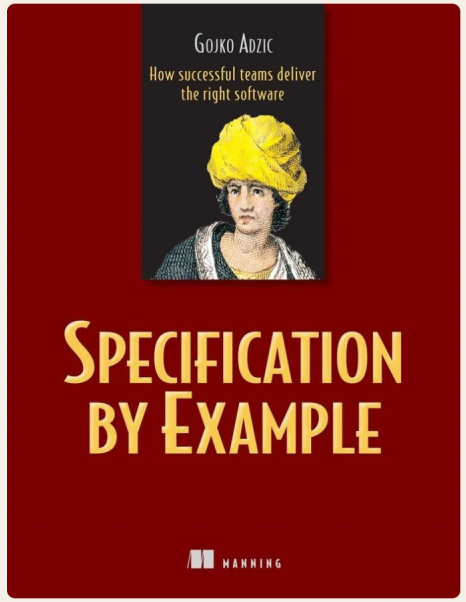
Domain-Driven Design

Eric Evans



Behaviour-Driven Development

Dan North



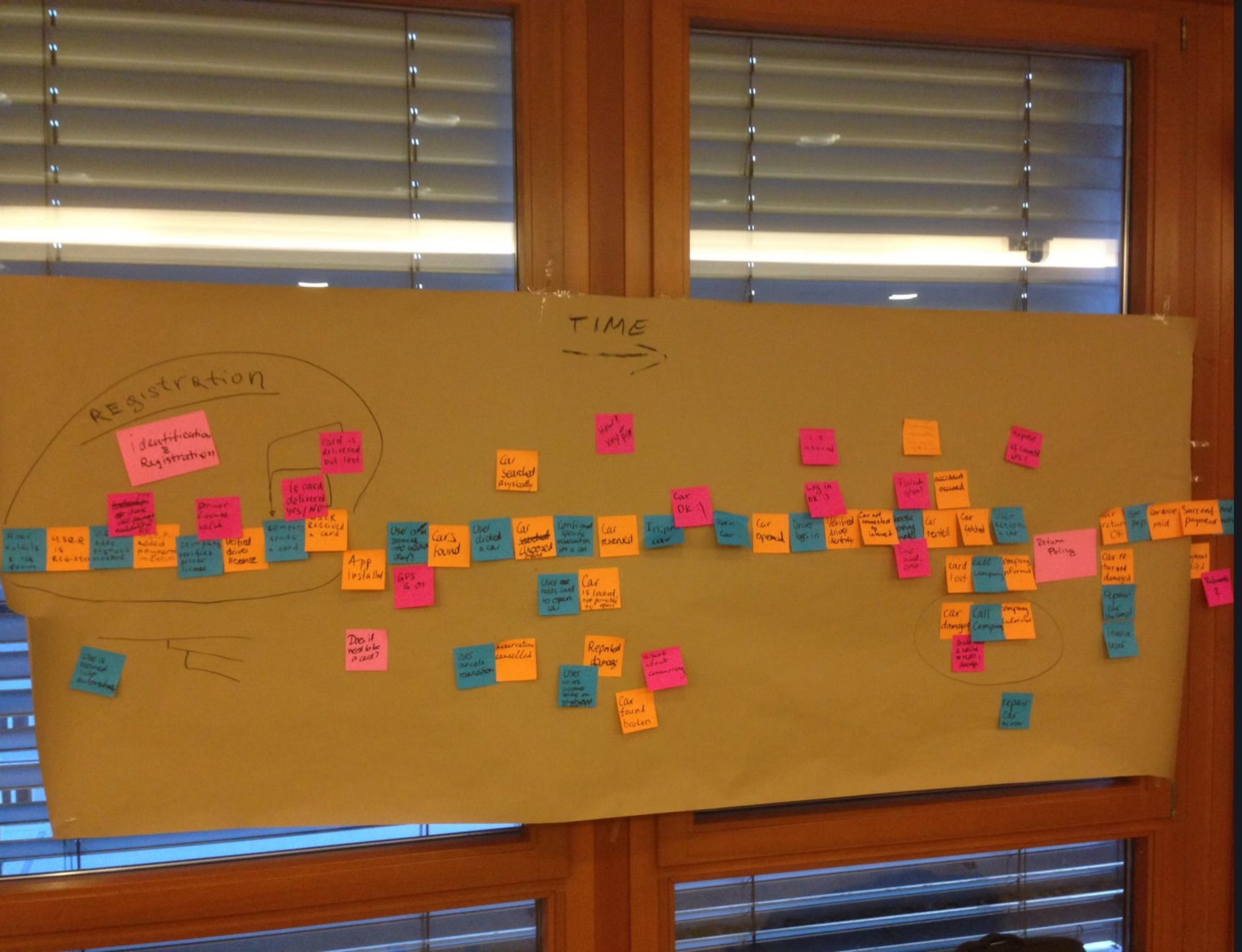
Specification by Example

Gojko Adzic


DOMAIN
DRIVEN
DESIGN
EUROPE
Alberto Brandolini



...and this is Lino, 11 years ago.



BOUNDED CONTEXT

...and a second concept?

Ubiquitous Language

A *formal* description of use cases.

LAST YEAR

My job as an architect



```
Ubiquitous Language =
{ business_requirements,
  functional_requirements }
```

LLM: f(Prompt, Ubiquitous Language) → Software

DEFINITION

Single source of truth

> *Every data element is mastered in only one place.*

Example: Protocol Buffers

How do Protocol Buffers Work?

The following diagram shows how you use protocol buffers to work with your data.

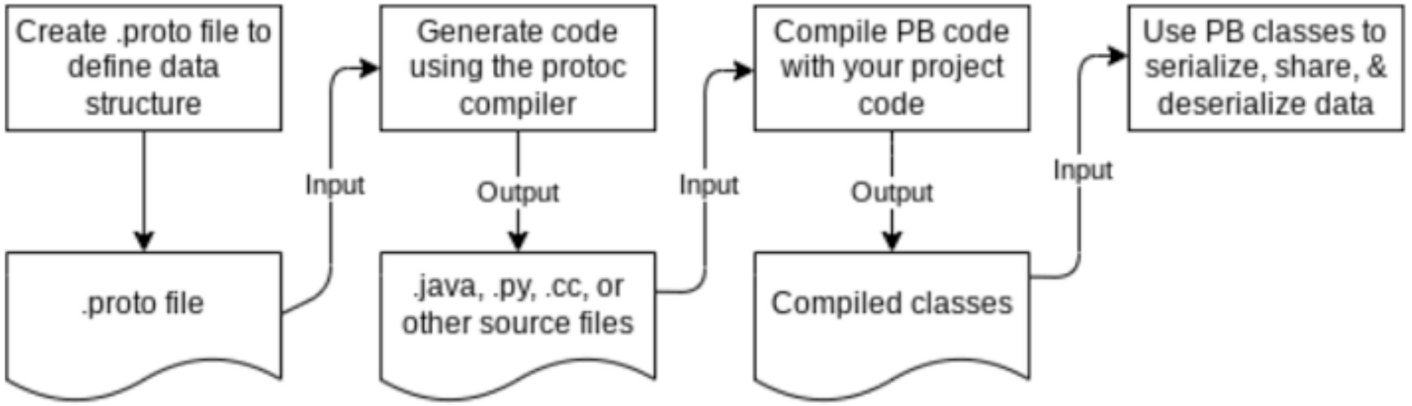


Figure 1. Protocol buffers workflow

You don't commit the generated code.

The pipeline regenerates it every time the .proto changes.



Our Ubiquitous Language becomes the system.

THE BENEFIT

A bug is fixed in the specification, never in the code.

Are LLMs deterministic?

Is the human brain?

PROOF BY CONSTRUCTION

$\mathbb{N} = \{0, 1, 2, \ldots\}$

Pick any x. I'll give you $x + 1$.

A system that consistently generates code that behaves the same way.

One year building that proof.

Take any source code.
Destroy it.
Rebuild it with an LLM.

Different code every time. The same behaviour every time.

THE PIPELINE

determ

analyse

steps 1–5 · reads the source

- clone
- profile
- harvest_behaviours
- augment_coverage
- freeze_api

strip

step 6 · the hard line

⚠ rm -rf src/

What crosses:

- tests
- behaviours
- api_skeleton
- project_profile

rebuild

steps 7–9 · never sees it

- write_requirements
- agents_rebuild
- validate

learn

steps 10–11

- genesis.md
- **conclude**

STEP 11

```
Ubiquitous Language = {
  behaviours,
  API description,
  functional requirements,
  tests
}
```

f(Ubiquitous Language)
↔ F(Programming Language)

Every program can be described without its source code.

WHERE THIS GOES

Delete the code.
Keep the language.



NEXT

g(language i) ↔ F(language j)

From any language to any other, human or programming.

LINEAGE

Getting to determinism

Unit tests

Not comparable

→

+ Frozen API

Still not comparable

→

+ BDD behaviours

The jump in quality

THE RULES

Keeping the experiment honest

1 No original source in any prompt once it's deleted.

2 No hand edits. Fix the pipeline, not the code.

3 No special cases.

Thank you.

Virgilio Lino

Why Ubiquitous Language is the Only Source Code You'll Commit in 2030